

Express + MongoDB Cheatsheet (Exam Ready)

Setup

1. Initialize Project

```
mkdir my-api
cd my-api
npm init -y
npm install express mongoose
```

2. Basic Server Structure

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();

// Middleware (MUST be BEFORE routes!)
app.use(express.json());

// Routes go here...

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Connect to Database

```
const mongoose = require('mongoose');

async function connectDB() {
  try {
    const MONGODB_URI = 'mongodb://localhost:27017/mydb';
    // Or Atlas: 'mongodb+srv://user:pass@cluster.mongodb.net/mydb'

    await mongoose.connect(MONGODB_URI);
    console.log('✅ Connected to MongoDB');
  } catch (error) {
    console.error('❌ MongoDB connection error:', error.message);
    process.exit(1);
  }
}

connectDB();
```

Mongoose Schema & Model

1. Define Schema

```
// models/Product.js
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true },
  category: { type: String, default: 'general' },
  inStock: { type: Boolean, default: true }
}, {
  timestamps: true // Adds createdAt and updatedAt
});

module.exports = mongoose.model('Product', productSchema);
```

2. Common Field Types

```
const schema = new mongoose.Schema({
  name: String,
  age: Number,
  isActive: Boolean,
  birthDate: Date,
  tags: [String],           // Array of strings
  profile: {                // Nested object
    bio: String,
    avatar: String
  }
});
```

3. Field Validation

```
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true
  },
  age: {
    type: Number,
    min: 18,
    max: 100
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  }
});
```

1. HTTP Methods

```
const express = require('express');
const app = express();

// GET - Retrieve data
app.get('/products', (req, res) => {
  res.json({ message: 'Get all products' });
});

// POST - Create data
app.post('/products', (req, res) => {
  res.status(201).json({ message: 'Product created' });
});

// PUT - Update data
app.put('/products/:id', (req, res) => {
  res.json({ message: 'Product updated' });
});

// DELETE - Delete data
app.delete('/products/:id', (req, res) => {
  res.json({ message: 'Product deleted' });
});
```

2. Route Parameters

```
// URL: /users/42
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.json({ userId });
});

// URL: /products/electronics/laptops
app.get('/products/:category/:type', (req, res) => {
  const { category, type } = req.params;
  res.json({ category, type });
});
```

3. Query Parameters

```
// URL: /search?q=laptop&sort=price&limit=10
app.get('/search', (req, res) => {
  const query = req.query.q;
  const sort = req.query.sort;
  const limit = req.query.limit;
  res.json({ query, sort, limit });
});

// URL: /products?category=phones&minPrice=500
app.get('/products', (req, res) => {
  const { category, minPrice } = req.query;
  res.json({ category, minPrice });
});
```

Middleware

1. Built-in Middleware

```
const express = require('express');
const app = express();

// Parse JSON request body (REQUIRED for POST/PUT)
app.use(express.json());

// Serve static files from 'public' folder
app.use(express.static('public'));
```

2. Custom Middleware

```
// Logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // MUST call next() to continue
});

// Authentication middleware
app.use((req, res, next) => {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  next();
});
```

3. Middleware Order (CRITICAL!)

```
const express = require('express');
const app = express();

// 1. Middleware FIRST (parsing, logging, etc.)
app.use(express.json());
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// 2. Routes AFTER middleware
app.get('/products', (req, res) => {
  res.json([{ name: 'Laptop' }]);
});

// 3. 404 handler LAST (catches unmatched routes)
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

app.listen(3000);
```

1. CREATE - Insert Data

```
const Product = require('./models/Product');

// Create single document
app.post('/api/products', async (req, res) => {
  try {
    const { name, price, category } = req.body;

    const product = await Product.create({
      name,
      price,
      category
    });

    res.status(201).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// Alternative: new + save
app.post('/api/products', async (req, res) => {
  try {
    const product = new Product(req.body);
    await product.save();

    res.status(201).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

2. READ - Retrieve Data

```
// Get all documents
app.get('/api/products', async (req, res) => {
  try {
    const products = await Product.find();
    res.status(200).json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Get with filter
app.get('/api/products', async (req, res) => {
  try {
    const products = await Product.find({
      category: 'electronics',
      price: { $gte: 100 } // price >= 100
    });
    res.status(200).json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Get one by ID
app.get('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json(product);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Get one with filter
app.get('/api/products/search', async (req, res) => {
  try {
    const product = await Product.findOne({ name: 'Laptop' });

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json(product);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```


3. UPDATE - Modify Data

```
// Update by ID
app.put('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,          // Return updated document
        runValidators: true // Run schema validation
      }
    );

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// Update with filter
app.put('/api/products/update', async (req, res) => {
  try {
    const result = await Product.updateOne(
      { name: 'Laptop' },
      { $set: { price: 999 } }
    );

    res.status(200).json(result);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

4. DELETE - Remove Data

```
// Delete by ID
app.delete('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json({ message: 'Product deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Delete with filter
app.delete('/api/products/delete', async (req, res) => {
  try {
    const result = await Product.deleteOne({ name: 'Laptop' });

    res.status(200).json({ message: 'Product deleted' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Query Methods

Find Operations

```
// Find all
const products = await Product.find();

// Find with filter
const electronics = await Product.find({ category: 'electronics' });

// Find with multiple conditions (AND)
const items = await Product.find({
  category: 'electronics',
  price: { $gte: 100 }
});

// Find one (first match or null)
const product = await Product.findOne({ name: 'Laptop' });

// Find by ID
const product = await Product.findById('507f1f77bcf86cd799439011');
```

Query Operators

```
// Comparison operators
await Product.find({ price: { $gt: 100 } }); // Greater than
await Product.find({ price: { $gte: 100 } }); // Greater than or equal
await Product.find({ price: { $lt: 500 } }); // Less than
await Product.find({ price: { $lte: 500 } }); // Less than or equal
await Product.find({ price: { $ne: 100 } }); // Not equal

// Logical operators
await Product.find({
  $or: [
    { category: 'electronics' },
    { category: 'books' }
  ]
});

await Product.find({
  $and: [
    { price: { $gte: 100 } },
    { inStock: true }
  ]
});
```

Chaining Methods

```
// Select specific fields (return only name and price)
const products = await Product.find()
  .select('name price');

// Exclude fields (return all except __v)
const products = await Product.find()
  .select('-__v');

// Sort (1 = ascending, -1 = descending)
const products = await Product.find()
  .sort({ price: -1 }); // Highest price first

// Limit results
const products = await Product.find()
  .limit(10);

// Skip (pagination)
const products = await Product.find()
  .skip(10)
  .limit(10); // Page 2 (skip first 10, get next 10)

// Combine multiple operations
const products = await Product.find({ category: 'electronics' })
  .select('name price')
  .sort({ price: -1 })
  .limit(5);
```

```
const express = require('express');
const mongoose = require('mongoose');
const Product = require('./models/Product');

const app = express();

// Middleware
app.use(express.json());

// Logger
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// Connect to MongoDB
async function connectDB() {
  try {
    await mongoose.connect('mongodb://localhost:27017/mydb');
    console.log('✅ Connected to MongoDB');
  } catch (error) {
    console.error('❌ Error:', error.message);
    process.exit(1);
  }
}
connectDB();

// GET all products (with optional filters)
app.get('/api/products', async (req, res) => {
  try {
    let filter = {};

    if (req.query.category) {
      filter.category = req.query.category;
    }

    if (req.query.minPrice) {
      filter.price = { $gte: parseFloat(req.query.minPrice) };
    }

    const products = await Product.find(filter);
    res.status(200).json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// GET one product by ID
app.get('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }
  }
});
```

```

    }

    res.status(200).json(product);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// POST create new product
app.post('/api/products', async (req, res) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// PUT update product
app.put('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// DELETE product
app.delete('/api/products/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);

    if (!product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    res.status(200).json({ message: 'Product deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// 404 handler (MUST be last!)
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

// Start server

```

```
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

Quick Tips

HTTP Status Codes

- `200` - OK (successful GET, PUT, DELETE)
- `201` - Created (successful POST)
- `400` - Bad Request (validation error)
- `404` - Not Found (resource doesn't exist)
- `500` - Internal Server Error (server/database error)

Error Handling Pattern

```
app.get('/api/products/:id', async (req, res) => {  
  try {  
    // Your code here  
    const product = await Product.findById(req.params.id);  
  
    if (!product) {  
      return res.status(404).json({ error: 'Not found' });  
    }  
  
    res.status(200).json(product);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

Request/Response Objects

```
// Request (req)  
req.params.id      // Route parameters  
req.query.search   // Query parameters  
req.body           // POST/PUT body data  
req.headers        // Request headers  
  
// Response (res)  
res.json(data)      // Send JSON  
res.status(200).json(data) // Send JSON with status  
res.send('text')    // Send text  
res.status(404).json({ error: 'msg' }) // Error response
```

Middleware Best Practices

```
// ✅ Correct order
app.use(express.json()); // 1. Built-in middleware
app.use(customLogger); // 2. Custom middleware
app.get('/products', ...); // 3. Routes
app.use(notFoundHandler); // 4. 404 handler (last!)

// ❌ Wrong order
app.get('/products', ...); // Route defined first
app.use(express.json()); // Middleware won't apply to route above!
```

Common Patterns

```
// Dynamic filter from query params
app.get('/api/products', async (req, res) => {
  try {
    let filter = {};
    if (req.query.category) filter.category = req.query.category;
    if (req.query.minPrice) filter.price = { $gte: req.query.minPrice };

    const products = await Product.find(filter);
    res.json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Pagination
app.get('/api/products', async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;
    const skip = (page - 1) * limit;

    const products = await Product.find()
      .skip(skip)
      .limit(limit);

    res.json(products);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Update specific fields only
app.put('/api/products/:id', async (req, res) => {
  try {
    const updates = {};
    if (req.body.price) updates.price = req.body.price;
    if (req.body.name) updates.name = req.body.name;

    const product = await Product.findByIdAndUpdate(
      req.params.id,
      updates,
      { new: true }
    );

    res.json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```


Mongoose Methods Summary

```
// CREATE
await Model.create(data)           // Create one
await Model.create([data1, data2]) // Create multiple
const doc = new Model(data); await doc.save()

// READ
await Model.find()                 // Find all
await Model.find(filter)           // Find with filter
await Model.findOne(filter)        // Find first match
await Model.findById(id)           // Find by ID

// UPDATE
await Model.findByIdAndUpdate(id, data, options)
await Model.findOneAndUpdate(filter, data, options)
await Model.updateOne(filter, data)

// DELETE
await Model.findByIdAndDelete(id)
await Model.findOneAndDelete(filter)
await Model.deleteOne(filter)
await Model.deleteMany(filter)
```

API with Query Filters

```
app.get('/api/books', async (req, res) => {
  try {
    let filter = {};

    if (req.query.author) {
      filter.author = req.query.author;
    }

    if (req.query.minPrice) {
      filter.price = { $gte: parseFloat(req.query.minPrice) };
    }

    if (req.query.category) {
      filter.category = req.query.category;
    }

    const books = await Book.find(filter)
      .sort({ createdAt: -1 })
      .limit(20);

    res.status(200).json(books);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Protected Route Pattern

```
// Middleware to check authentication
function requireAuth(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  // Verify token here...
  next();
}

// Apply to specific route
app.delete('/api/products/:id', requireAuth, async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    res.json({ message: 'Deleted' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Validation Pattern

```
app.post('/api/products', async (req, res) => {
  try {
    const { name, price, category } = req.body;

    // Manual validation
    if (!name || !price) {
      return res.status(400).json({
        error: 'Name and price are required'
      });
    }

    if (price < 0) {
      return res.status(400).json({
        error: 'Price must be positive'
      });
    }

    const product = await Product.create({ name, price, category });
    res.status(201).json(product);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```